

GlobalBridge

An Integrated, AI-Assisted Web Platform for International Student and Immigrant Support

Final Year Project Report
Chapters 1 - 3

Name	Index Number	Responsibility
Eric Asante	3376122	Backend, Database & Deployment
Baddoo Jeremiah Nii Adotei	3381622	Frontend & UI/UX Design

Group 8 - Academic Year 2025/2026

Table of Contents

Chapter 1: Introduction

- 1.1 Problem Statement
- 1.2 Aim of the Project
- 1.3 Specific Objectives
- 1.4 Justification
- 1.5 Motivation
- 1.6 Scope
- 1.7 Project Limitations
- 1.8 Beneficiaries
- 1.9 Academic and Practical Relevance
- 1.10 Activity Planning and Schedule
- 1.11 Structure of Report
- 1.12 Project Deliverables

Chapter 2: Review of Related Works

- 2.1 Existing Systems (Features, Pros and Cons)
- 2.2 The Proposed System
- 2.3 Conceptual Design
- 2.4 Architecture of the Proposed System
- 2.5 Component Designs and Descriptions
- 2.6 Proposed System Features
- 2.7 Development Tools and Environment
- 2.8 Benefits of the Proposed System

Chapter 3: Methodology

- 3.1 Chapter Overview
- 3.2 Requirement Specification
- 3.3 Stakeholders
- 3.4 Requirement Gathering Process
- 3.5 Functional Requirements
- 3.6 UML Diagrams
- 3.7 Non-Functional Requirements
- 3.8 Security Concepts
- 3.9 Project Methods
- 3.10 Software Process Models
- 3.11 Chosen Model and Justification
- 3.12 Design Considerations (UI & Database)
- 3.13 Development Tools

References

Chapter 1: Introduction

1.1 Problem Statement

Every year millions of students and professionals relocate across borders to study, work, or settle. Yet the information and services they depend on remain fragmented, unreliable, and often predatory. Prospective migrants must navigate constantly changing visa rules scattered across government portals, locate trustworthy accommodation from afar while exposed to rental scams, find mentors and employers who understand visa sponsorship, and overcome language and cultural barriers, all while managing day-to-day logistics such as banking, healthcare and transport in an unfamiliar country. Existing solutions are siloed: official immigration websites are authoritative but hard to interpret, social-media groups are informal and unverified, and paid agents are expensive and inconsistent. There is no single trustworthy platform that unifies visa guidance, verified housing, mentorship, opportunities and practical settlement support for international students and immigrants.

1.2 Aim of the Project

The aim of this project is to design and implement **GlobalBridge**, an integrated, secure and AI-assisted web platform that consolidates visa guidance, a verified housing marketplace, mentorship, scholarship and job opportunities, community support and a life-support toolkit into a single, role-aware experience for international students, immigrants, mentors and employers.

1.3 Specific Objectives of the Project

The specific objectives are:

- 1 To develop a role-based authentication system supporting students, mentors, employers and administrators using Firebase Authentication with custom role claims.
- 2 To implement an AI-powered visa assistant that provides contextual guidance, generates document checklists and performs document checks using a large language model.
- 3 To build a verified opportunities board (scholarships, internships, work-study and jobs) with visa-sponsorship awareness.
- 4 To provide a housing marketplace with listing management and safeguards against fraudulent listings.
- 5 To enable structured mentorship through mentor profiles, availability and session bookings.
- 6 To create community features (forums, success stories and real-time messaging) that reduce isolation.
- 7 To deliver an administrative control centre for verification, moderation, content management and platform analytics.
- 8 To deploy the system on managed cloud infrastructure and evaluate it against functional and non-functional requirements.

1.4 Justification of Project

A unified platform is justified because the cost of fragmentation falls hardest on a vulnerable population. Rental and admission scams cause real financial loss; misinterpreted visa rules lead to rejected applications; and isolation harms wellbeing and retention. By consolidating verified information and trustworthy services, GlobalBridge reduces risk, lowers reliance on costly

intermediaries, and shortens the time it takes a newcomer to become settled and productive. The use of AI further democratises access to guidance that would otherwise require expensive professional consultation.

1.5 Motivation for Undertaking the Project

The project is motivated by the team's first-hand exposure to the difficulties faced by international students, and by the observation that modern web and AI technologies are now mature enough to deliver, at low cost, the kind of personalised guidance that was previously unavailable. The opportunity to apply full-stack engineering, cloud deployment and applied AI to a problem with clear social value provided strong motivation to undertake the project.

1.6 Scope of Project

GlobalBridge is a responsive web application comprising a Next.js frontend, an Express REST and WebSocket API, a FastAPI AI microservice and a PostgreSQL database. Within scope are: account management and role-based access; the AI visa assistant, checklist generator and document checker; the opportunities, housing, mentorship, jobs and community modules; the life-support toolkit; and the administrative dashboard. Out of scope are native mobile applications, real payment settlement, and automated submission of applications to third-party immigration authorities; these are identified as future work.

1.7 Project Limitations

- The AI assistant provides guidance for informational purposes and is not a substitute for licensed legal advice.
- Free-tier cloud hosting introduces cold-start latency on the backend after periods of inactivity.
- Payment and document storage are abstracted but not connected to live commercial providers in this version.
- Listing verification combines automated checks with manual review and therefore depends on administrator availability.
- External data (visa rules, exchange rates) reflects the state captured at the time of use and may change.

1.8 Beneficiaries of the Project

- **International students and immigrants** - the primary users, who gain consolidated, trustworthy guidance and services.
- **Mentors** - experienced migrants and professionals who guide newcomers and build community impact.
- **Employers** - organisations seeking international talent and offering visa sponsorship.
- **Educational institutions and support offices** - which can direct students to a reliable resource.
- **Administrators and moderators** - who maintain platform integrity and safety.

1.9 Academic and Practical Relevance of the Project

Academically, the project demonstrates the application of software-engineering principles - requirements engineering, architectural design, UML modelling, agile delivery and testing - to a

non-trivial, multi-service system, and the integration of large-language-model technology into a production web application. **Practically**, it produces a deployable product that addresses a genuine social need, exercising modern industry tools (Next.js, Express, FastAPI, PostgreSQL, Firebase) and cloud platforms (Vercel, Render, Neon).

1.10 Project Activity Planning and Schedules

The project followed an iterative, agile schedule across the academic year. The major activities and their indicative timeline are summarised below.

#	Activity	Timeline
1	Problem definition & requirements gathering	Weeks 1-3
2	System & database design (UML, ER, wireframes)	Weeks 3-5
3	Authentication & role-based access control	Weeks 5-7
4	AI service (visa assistant, checklist, doc-check)	Weeks 7-9
5	Core modules (opportunities, housing, mentorship, jobs)	Weeks 9-13
6	Community, messaging & toolkit	Weeks 13-15
7	Admin dashboard, moderation & analytics	Weeks 15-17
8	Testing, deployment & documentation	Weeks 17-20

Table 1.1 - Project activity plan and schedule

1.11 Structure of Report

Chapter 1 introduces the problem, aim, objectives and scope. **Chapter 2** reviews existing systems, presents the proposed system, its conceptual and architectural design, components, features and tooling. **Chapter 3** details the methodology: requirements, stakeholders, UML models, non-functional and security requirements, the chosen process model and the logical (UI and database) design.

1.12 Project Deliverables

- A deployed, responsive web application with role-based dashboards for students, mentors, employers and administrators.
- An Express REST + WebSocket API and a FastAPI AI microservice.
- A normalised PostgreSQL database with schema and seed data.
- An AI visa assistant, checklist generator, document checker and translation service.
- Project documentation including requirements, UML models, design artefacts and a user/setup guide.
- Automated tests and continuous deployment to Vercel and Render.

Chapter 2: Review of Related Works

2.1 Processes of the Existing Systems

Newcomers currently assemble support from several disconnected sources. Each addresses one slice of the problem and exhibits characteristic strengths and weaknesses, summarised below.

Existing System	Pros	Cons
Government immigration portals	Authoritative, official, free	Hard to interpret; no personalisation; no housing/mentorship
University international offices	Trusted; institution-specific help	Limited hours; scope confined to enrolled students
Social media groups (Facebook/WhatsApp)	Active, community-driven, free	Unverified, scam-prone, unstructured, no privacy controls
Immigration agents/consultants	Personalised expert help	Expensive; inconsistent quality; not scalable
Generic housing sites (e.g. classifieds)	Large inventory	No newcomer focus; high fraud risk; no verification
Generic job boards	Wide reach	No visa-sponsorship filter; not tailored to migrants

Table 2.1 - Comparison of existing related systems

2.2 The Proposed System

The proposed system, GlobalBridge, unifies these fragmented services into a single, trustworthy, role-aware web platform. It augments human and community support with an AI visa assistant, enforces verification and moderation to combat fraud, and presents each user with a dashboard tailored to their role. The platform is built as a set of loosely-coupled modules sharing one identity and data layer, allowing each capability to evolve independently while delivering a coherent experience.

2.3 Conceptual Design

Conceptually, GlobalBridge is organised around four actors (student, mentor, employer, administrator) interacting with eight functional domains: identity, AI guidance, opportunities, housing, mentorship, jobs, community and administration. A single sign-on identity (Firebase) issues a verifiable token carrying a role claim; this claim drives both client-side navigation and server-side authorisation, ensuring each actor sees and can act only on the capabilities relevant to them. All modules read from and write to a shared relational store, with an AI microservice providing language intelligence on demand.

2.4 Architecture of the Proposed System

GlobalBridge adopts a three-tier architecture augmented by a dedicated AI microservice. The presentation tier is a server-rendered Next.js application [4]; the application tier is an Express REST and WebSocket API [8], [17]; the data tier is a managed PostgreSQL database [14] with an optional Redis cache [15]. Language intelligence is isolated in a FastAPI microservice [10] so that AI workloads scale and fail independently of the core API.

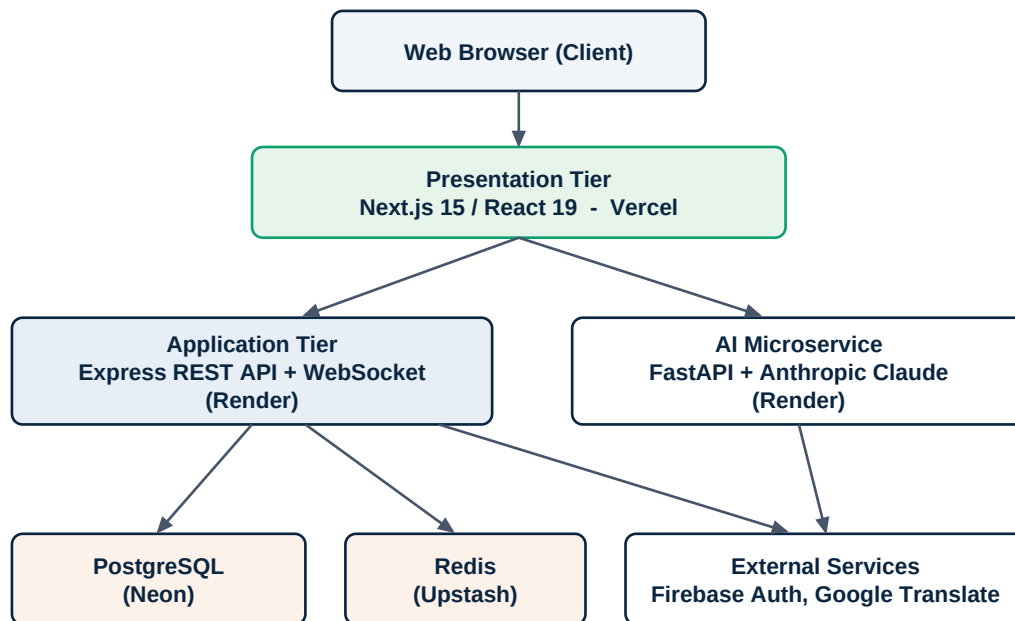


Figure 2.1 - Three-tier architecture with an AI microservice

2.5 Components Designs and Component Descriptions

The system is composed of the following principal components. Each exposes a well-defined interface and can be understood and tested independently.

Authentication & Authorisation Component

Handles registration, sign-in and session continuity. The frontend uses the Firebase client SDK [12] to create accounts and obtain a short-lived ID token; the backend verifies this token with the Firebase Admin SDK, resolves the user to a PostgreSQL record and attaches a role claim. A guard mechanism on the client routes each role to its dashboard and blocks cross-role access, while every API route re-verifies the token and role on the server. This dual enforcement ensures that navigation convenience never weakens real security.

AI Guidance Component

A FastAPI microservice exposes endpoints for conversational visa guidance, checklist generation, document checking and translation. It composes a structured prompt from the user's origin and destination countries and visa type, sends it to the Anthropic Claude model [11] with prompt caching to control cost, and returns a structured reply with source hints. The checklist generator derives an ordered list of required documents, and the document checker evaluates submitted text against expected criteria. Translation is delegated to the Google Translate API [13].

Opportunities & Jobs Component

Manages scholarships, internships, work-study placements and jobs as records of a single 'opportunities' entity typed by category. Employers post roles, optionally flagged as visa-sponsoring; students search, filter and save opportunities. View counts and saved-item signals feed analytics and

recommendations.

Housing Component

Provides creation, browsing and detail views of accommodation listings, including roommate preferences. Listings carry a status (draft, pending review, active, rented, archived) so that administrators can verify them before they go live, directly mitigating the rental-scam risk identified in Chapter 1.

Mentorship Component

Maintains mentor profiles (expertise, languages, availability) and student-mentor bookings with date, time, duration and goal. Aggregated metrics - active mentees, sessions and community contributions - power the mentor dashboard.

Community & Messaging Component

Implements forums (categories, posts, replies, accepted answers), success stories and real-time direct messaging over WebSocket. When Redis is configured it provides pub/sub fan-out; otherwise the system degrades gracefully to a single instance, preserving functionality in constrained deployments.

Administration & Moderation Component

Offers a control centre for user management, verification of mentors, employers and listings, handling of reported content and scam alerts, and platform analytics (user counts by role, daily sign-ups, content and AI usage). Access is restricted to the administrator role at both the route and API levels.

2.6 Proposed System / Software Features

- Role-based authentication and dashboards for students, mentors, employers and administrators.
- AI visa assistant with checklist generation and document checking.
- Verified opportunities board with visa-sponsorship awareness.
- Housing marketplace with listing verification and roommate matching.
- Structured mentorship with profiles and session booking.
- Community forums, success stories and real-time messaging.
- Life-support toolkit (banking, healthcare, SIM, transit, tax, emergency SOS and more).
- Administrative verification, moderation and analytics.
- Multilingual support through on-demand translation; responsive, dark-mode-aware UI.

2.7 Development Tools and Environment

The system is implemented with the following tools and environment, described in greater operational detail in Chapter 3.

Layer	Tools / Technologies
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS v4
Backend API	Node.js, Express, WebSocket (ws), TypeScript, Zod
AI Service	Python, FastAPI, Anthropic Claude SDK, Google Cloud Translate
Database & Cache	PostgreSQL (Neon), Redis (Upstash, optional)

Authentication	Firebase Authentication & Firebase Admin SDK
Hosting & CI/CD	Vercel (frontend), Render (backend & AI), GitHub
Tooling	Git/GitHub, Vitest, Docker, ESLint/TypeScript compiler

Table 2.2 - Development tools and environment

2.8 Benefits of Implementation of the Proposed System

- Consolidation of fragmented services into one trustworthy platform, saving time and reducing risk.
- Lower cost of guidance through AI, reducing dependence on expensive agents.
- Reduced exposure to housing and admission fraud through verification and moderation.
- Reduced isolation through mentorship and community features.
- Scalable, independently deployable services with graceful degradation.
- A foundation that institutions and employers can adopt to reach international audiences.

Chapter 3: Methodology

3.1 Chapter Overview

This chapter describes how the system was engineered. It presents the requirement-specification process, the stakeholders and how their needs were gathered, the functional and non-functional requirements, the UML models that capture system behaviour and structure, the security concepts, the chosen software-process model and its justification, and the logical (user-interface and database) design.

3.2 Requirement Specification

Requirements were elicited and specified iteratively. Each requirement was recorded, classified as functional or non-functional, prioritised, and validated against the project aim. The specification was refined at the end of each iteration as understanding deepened and as working software exposed implicit needs.

3.3 Stakeholders of the System

The principal stakeholders are:

- **Students / immigrants** - primary end users seeking guidance and services.
- **Mentors** - provide guidance and answer community questions.
- **Employers** - post jobs and seek international talent.
- **Administrators** - verify, moderate and monitor the platform.
- **Project supervisor and academic assessors** - evaluate the work.
- **The development team** - design, build, test and maintain the system.

3.4 Requirement Gathering Process

Requirements were gathered through a combination of techniques: review of existing systems and official immigration resources to understand current processes and gaps; informal interviews and conversations with international students about their pain points; analysis of comparable platforms to identify desirable features; and iterative prototyping, in which working increments were demonstrated and feedback fed back into the next iteration.

3.5 Functional Requirements

The system shall provide the following functional capabilities:

- 1 The system shall allow users to register and authenticate, selecting a role of student, mentor or employer.
- 2 The system shall route each authenticated user to a role-specific dashboard and prevent access to other roles' areas.
- 3 The system shall let students obtain AI visa guidance and generate document checklists.
- 4 The system shall allow users to search, filter, view and save opportunities (scholarships, internships, jobs).
- 5 The system shall allow employers to post jobs and flag visa sponsorship.

- 6 The system shall allow users to browse and list housing, subject to verification.
- 7 The system shall allow students to view mentor profiles and book mentorship sessions.
- 8 The system shall provide forums, success stories and real-time messaging.
- 9 The system shall allow users to report content and shall surface scam alerts.
- 10 The system shall allow administrators to verify users and listings, moderate content and view platform analytics.

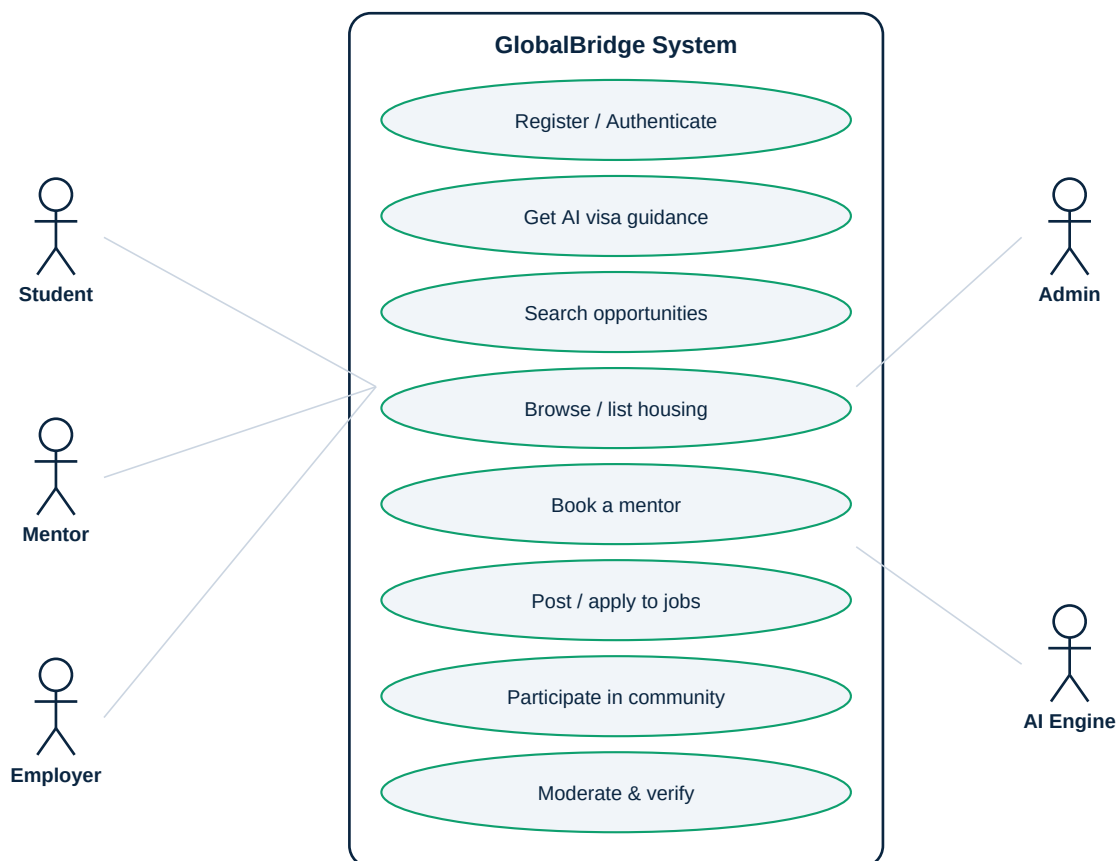
3.6 UML Diagrams

The following UML models capture the behaviour and structure of GlobalBridge.

3.6.1 Use Case Diagram

The use case diagram below shows the principal actors and the use cases they participate in. The frontend (client) use cases are those initiated by human actors, while the back-end use cases (token verification, persistence, AI inference) are realised by the API and the AI engine in support of them.

Figure 3.1 - Use case diagram (front-end and back-end actors)



3.6.2 Use Case Descriptions

The roles of the actors are: the **Student** seeks guidance, opportunities, housing, mentorship and community; the **Mentor** offers guidance and answers questions; the **Employer** posts jobs and reviews interested candidates; the **Administrator** verifies, moderates and monitors; and the **AI Engine** is a supporting actor that performs language inference. Representative use cases are

described below.

Use Case	Actor(s)	Description
Register / Authenticate	Student/Mentor/Employer	User creates an account or signs in; the system verifies the identity token, provisions a user record and assigns a role claim.
Get AI visa guidance	Student, AI Engine	Student asks a visa question; the system builds a prompt and returns AI guidance with a generated document checklist.
Search opportunities	Student	Student searches and filters scholarships and jobs and saves items of interest.
Book a mentor	Student, Mentor	Student selects a mentor and books a session; the mentor reviews and confirms the request.
List / verify housing	Mentor/Employer, Administrator	A listing is created in a pending state and becomes active only after administrator verification.
Moderate & verify	Administrator	Administrator reviews reports, verifies users and listings, and monitors analytics.

Table 3.1 - Use case descriptions

3.6.3 Activity Diagram (Registration & Routing)

The activity flow for registration is: the user submits the sign-up form; the system creates a Firebase account and obtains an ID token; the backend verifies the token and upserts the user record with a role claim; on success the user is redirected to the dashboard for their role; on failure an error is shown and the user may retry. Decision points handle duplicate accounts and invalid credentials.

3.6.4 Sequence Diagram (Registration & Authentication)

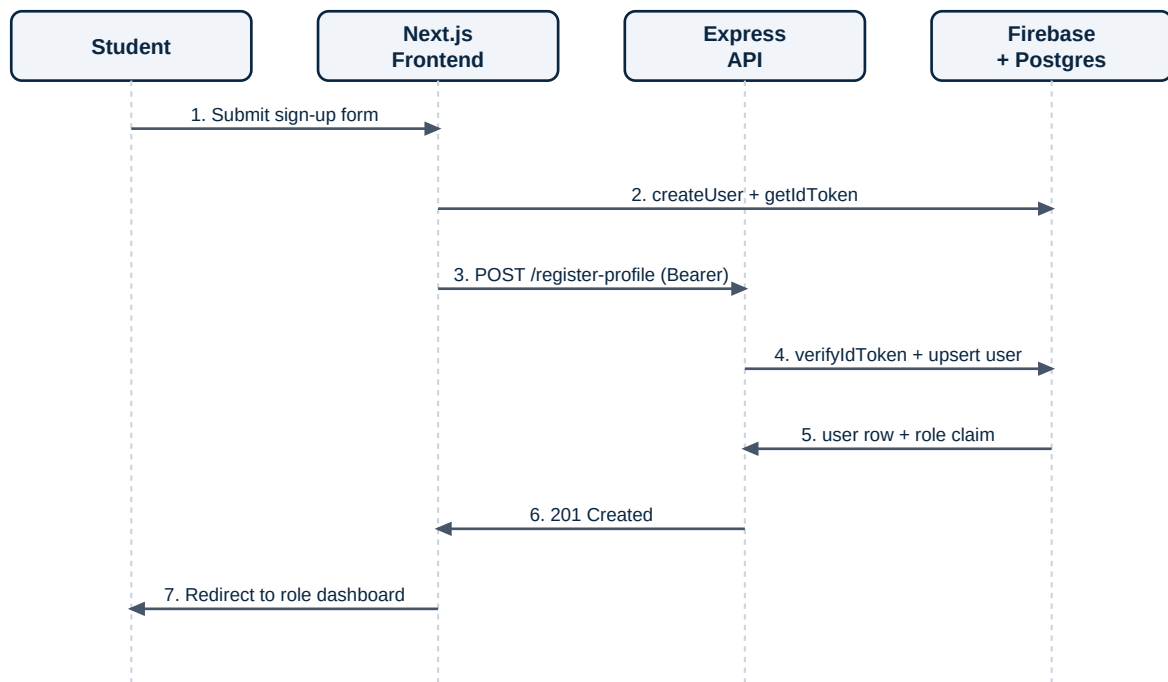


Figure 3.3 - Sequence diagram: student registration and authentication

3.6.5 Class Diagram (Core Domain)

The core domain classes and their principal relationships are summarised below. A User has one role and may own many Opportunities, Bookings, SavedItems, ForumPosts and Messages; Opportunities are saved by many Users; Bookings associate a student User with a mentor User.

Class	Key Attributes	Relationships
User	id, email, role, fullName, countryOfOrigin, verificationStatus	owns Opportunities, Bookings, SavedItems, Posts
Opportunity	id, type, title, country, deadline, sponsorsVisa, viewCount	posted by User; saved by Users
MentorBooking	id, mentorId, studentId, slotDate, slotTime, status	links student User and mentor User
HousingListing	id, title, status, rating, location	posted by User; verified by Admin
VisaChecklist	id, userId, destination, items, completedItems	belongs to User
ForumPost	id, authorId, title, body, answerCount	has many ForumReplies

Table 3.2 - Core domain classes (class diagram in tabular form)

3.7 Non-Functional Requirements

The non-functional requirements, with justifications, are as follows.

Requirement	Statement	Justification
-------------	-----------	---------------

Security	User data and credentials must be protected and access must be role-appropriate.	The platform handles personal and verification data for a vulnerable population.
Usability	The interface must be clear, responsive and accessible, with dark-mode support.	Users vary in device, language and digital literacy.
Performance	Common interactions should respond within a few seconds under normal load.	Slow responses deter use and harm trust.
Reliability	The system should degrade gracefully and tolerate backend cold starts.	Free-tier hosting and optional services must not cause failures.
Scalability	Components must scale independently.	AI and core workloads have different load profiles.
Maintainability	Code must be modular, typed and tested.	The system will evolve across iterations and contributors.
Portability	The system must run on managed cloud platforms via standard runtimes/containers.	Avoids vendor lock-in and eases deployment.

Table 3.3 - Non-functional requirements with justifications

3.8 Security Concepts

Because GlobalBridge stores personal and verification data, security is a first-class concern addressed at several layers, in line with established OWASP guidance [18]:

- **Authentication** - identities are managed by Firebase Authentication; the backend verifies each request's ID token with the Firebase Admin SDK before any protected action.
- **Authorisation** - a role claim (student, mentor, employer, admin) is enforced both by client-side route guards and by server-side checks on every API route, so privilege cannot be escalated by manipulating the client.
- **Transport security** - all traffic is served over HTTPS by the hosting platforms.
- **Input validation** - request payloads are validated with Zod schemas [16] and user-supplied strings are sanitised to prevent injection and cross-site scripting.
- **HTTP hardening** - the API applies Helmet security headers, Cross-Origin Resource Sharing restricted to the known frontend origin, and CSRF protection.
- **Rate limiting** - request rate limiting protects against brute-force and abuse.
- **Secret management** - service-account keys and credentials are held in environment variables, never in source control.
- **Content integrity** - listing verification, content moderation and scam alerts protect users from fraud.

3.9 Project Methods (Agile or Plan-Driven)

Two broad families of method were considered: plan-driven (predictive) approaches, which fix requirements early and proceed sequentially, and agile (adaptive) approaches, which embrace changing requirements through short, iterative cycles. Given that the team's understanding of user needs was expected to deepen over time and that working software was the best way to validate the concept, an agile method was adopted.

3.10 The Various Software Process Models

The following models were considered [1], [2]:

- **Waterfall** - sequential phases; simple to manage but inflexible to change and risky for evolving requirements.
- **Incremental** - delivers the system in increments; balances structure with partial early delivery.
- **Spiral** - risk-driven and iterative; powerful but heavyweight for a student project.
- **Prototyping** - builds throwaway or evolutionary prototypes to clarify requirements.
- **Agile / Scrum** - short iterations (sprints) with continuous feedback, suited to changing requirements and small teams.

3.11 Chosen Model and Justification

An **agile, iterative model in the spirit of Scrum** [3] was chosen. Work proceeded in short iterations, each producing a working, deployable increment (for example: authentication, then the AI service, then each functional module, then the dashboards and admin tools). This choice is justified because: requirements evolved as the team learned more about user needs; continuous deployment to Vercel and Render allowed each increment to be validated quickly; the small, two-person team benefited from lightweight process and frequent integration; and risk was reduced by delivering and testing functionality early rather than deferring integration to the end.

3.12 Project Design Consideration (Logical Designs)

3.12.1 User-Interface Design (Wireframes)

The interface follows a modern, trustworthy SaaS aesthetic with a deep-blue and emerald palette, clear typography, responsive layouts and dark-mode support. The principal screens were wireframed before implementation:

- **Authentication screen** - a split-screen layout pairing a branded trust panel (statistics, testimonial, security badges) with an accessible sign-in / multi-step sign-up form.
- **Student dashboard** - a welcome header, profile-completion indicator, key statistics, status trackers (visa, applications, housing), quick actions and recommendation widgets.
- **Mentor dashboard** - mentee and request counts, upcoming sessions, community-impact metrics and management actions.
- **Employer dashboard** - active listings, interested candidates, job views and a visa-sponsorship indicator.
- **Admin dashboard** - platform analytics, verification and moderation queues, and content management.

3.12.2 Database Design (ER Diagram & Schema)

The database is a normalised PostgreSQL schema. The simplified entity-relationship overview below shows the core entities and their relationships; the principal tables and their purpose follow.

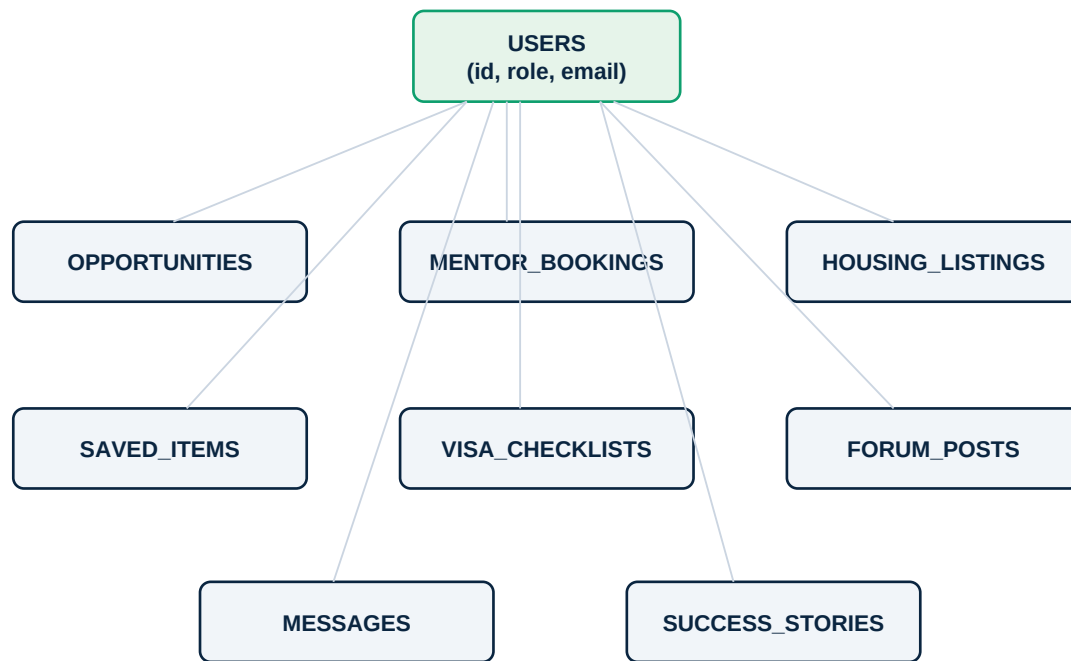


Figure 3.4 - Simplified entity-relationship overview (core entities)

Table	Purpose
users	Accounts, roles, profile and verification status (identity bridge to Firebase).
mentor_profiles / employer_profiles	Role-specific extended profiles.
opportunities	Scholarships, internships, work-study and jobs (typed), with visa-sponsorship flag.
housing_listings / roommate_preferences	Accommodation listings and roommate matching.
mentor_bookings	Student-mentor session bookings.
saved_items	User-saved opportunities and housing (interest signal).
visa_checklists	Per-user visa document checklists and completion state.
forum_categories / forum_posts / forum_replies	Community discussion structures.
conversations / messages	Real-time direct messaging.
success_stories	Verified migrant success stories.
notifications	Per-user notifications.
reports / scam_alerts	Moderation reports and fraud alerts.
ai_conversations / ai_messages	Persisted AI assistant conversations.

Table 3.4 - Principal database tables and their purpose

3.13 Developmental Tools

The development tools introduced in Chapter 2 were used as follows during implementation:

- **Next.js 15 & React 19 (TypeScript, Tailwind CSS v4)** [4], [5], [6], [7] - used to build the server-rendered, responsive frontend, with the App Router providing role-segmented routes and layouts, and client guards enforcing role-based navigation.
- **Node.js & Express (TypeScript)** [9], [8] - used to implement the REST API and route handlers, with middleware for authentication, CSRF, Helmet, CORS and rate limiting; **ws** provides WebSocket messaging [17].
- **Zod** [16] - used to validate and type-check all request payloads and environment configuration at runtime.
- **FastAPI (Python)** [10] - used to implement the AI microservice endpoints; the **Anthropic Claude SDK** [11] performs inference with prompt caching, and **Google Cloud Translate** [13] performs translation.
- **PostgreSQL (Neon)** [14], [21] - used as the primary relational store via the node-postgres driver and parameterised queries; **Redis (Upstash)** [15] optionally provides caching and pub/sub.
- **Firebase Authentication & Admin SDK** [12] - used for identity management and server-side token verification.
- **Git & GitHub** - used for version control and as the trigger for continuous deployment.
- **Vercel & Render** [19], [20] - used to host the frontend and the backend/AI services respectively, with automatic deployment on each push.
- **Vitest** [23] - used for automated unit testing of core logic; the **TypeScript compiler** [6] enforces type safety across the codebase.
- **Docker** [22] - used to containerise the backend for reproducible deployment.

References

The following sources, comprising official technical documentation, standards and software-engineering literature, informed the design and implementation of the GlobalBridge platform.

- [1] I. Sommerville, *Software Engineering*, 10th ed. Harlow, England: Pearson Education, 2015.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2015.
- [3] K. Schwaber and J. Sutherland, "The Scrum Guide," Scrum.org, 2020. [Online]. Available: <https://scrumguides.org>. [Accessed: 2026].
- [4] Vercel Inc., "Next.js Documentation (App Router)," 2025. [Online]. Available: <https://nextjs.org/docs>. [Accessed: 2026].
- [5] Meta Platforms Inc., "React Documentation," 2025. [Online]. Available: <https://react.dev>. [Accessed: 2026].
- [6] Microsoft Corporation, "TypeScript Documentation," 2025. [Online]. Available: <https://www.typescriptlang.org/docs>. [Accessed: 2026].
- [7] Tailwind Labs, "Tailwind CSS Documentation," 2025. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: 2026].
- [8] OpenJS Foundation, "Express - Node.js Web Application Framework," 2025. [Online]. Available: <https://expressjs.com>. [Accessed: 2026].
- [9] OpenJS Foundation, "Node.js Documentation," 2025. [Online]. Available: <https://nodejs.org/docs>. [Accessed: 2026].
- [10] S. Ramirez, "FastAPI Documentation," 2025. [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed: 2026].
- [11] Anthropic PBC, "Claude API Documentation," 2025. [Online]. Available: <https://docs.anthropic.com>. [Accessed: 2026].
- [12] Google LLC, "Firebase Authentication Documentation," 2025. [Online]. Available: <https://firebase.google.com/docs/auth>. [Accessed: 2026].
- [13] Google LLC, "Cloud Translation API Documentation," 2025. [Online]. Available: <https://cloud.google.com/translate/docs>. [Accessed: 2026].
- [14] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs>. [Accessed: 2026].
- [15] Redis Ltd., "Redis Documentation," 2025. [Online]. Available: <https://redis.io/docs>. [Accessed: 2026].
- [16] Colin McDonnell, "Zod: TypeScript-first Schema Validation," 2025. [Online]. Available: <https://zod.dev>. [Accessed: 2026].
- [17] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Internet Engineering Task Force, Dec. 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455>.
- [18] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: <https://owasp.org/Top10>. [Accessed: 2026].
- [19] Vercel Inc., "Vercel Platform Documentation," 2025. [Online]. Available: <https://vercel.com/docs>. [Accessed: 2026].
- [20] Render Services Inc., "Render Documentation," 2025. [Online]. Available: <https://render.com/docs>. [Accessed: 2026].

- [21] Neon Inc., "Neon Serverless Postgres Documentation," 2025. [Online]. Available: <https://neon.tech/docs>. [Accessed: 2026].
- [22] Docker Inc., "Docker Documentation," 2025. [Online]. Available: <https://docs.docker.com>. [Accessed: 2026].
- [23] Vitest, "Vitest - Next Generation Testing Framework," 2025. [Online]. Available: <https://vitest.dev>. [Accessed: 2026].